

openpv/simshady: A Javascript Package for Photovoltaic Yield Estimation Based on 3D Meshes

Florian Kotthoff^{1,2}, Mino Estrella³, Martin Großhauser¹, Konrad Heidler¹, and Korbinian Pöppel¹

¹ OpenPV GbR, Munich, Germany ² OFFIS Institute for Information Technology, Escherweg 2, 26121 Oldenburg, Germany ³ Chair of Renewable and Sustainable Energy Systems, Technical University of Munich, Germany ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Adam R. Jensen](#)

Reviewers:

- [@RDaxini](#)
- [@kbonney](#)

Submitted: 12 July 2025

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

openpv/simshady (short: simshady) is a JavaScript package for simulating photovoltaic (PV) energy yields. It integrates local climate data and 3D objects into its shading simulation, utilizing Three.js meshes for geometric modeling. The package performs shading analysis using a WebGL-parallelized implementation of the Möller-Trumbore intersection algorithm (Möller & Trumbore, 1997). With the software, researchers can obtain both aggregated and spatial or temporally resolved simulated scenes of expected PV yields for further processing. Simshady provides both a package for web development, as well as a Command-line interface (CLI) tool for simulations on your machine.

Statement of need

To meet global climate targets, solar photovoltaic (PV) capacity must expand significantly. Tripling renewable energy capacity by 2030 is essential to limit global warming to 1.5°C (International Energy Agency, 2023). The expansion of PV plays a crucial role, and PV systems offer an additional benefit: small-scale house-mounted PV systems enable public participation and legitimize the energy transition. For calculating the yield of PV systems, various factors are important, including the location of the planned installation, local climate, surrounding objects such as houses or trees, and terrain. To provide accurate estimates of expected yields, simulation tools are essential in both research and practical PV system planning. For these reasons, a variety of software tools for simulating photovoltaic systems already exist (Holmgren, Hansen, Stein, et al., 2018; Jakica, 2018; Kumar Vashishtha et al., 2022). Simshady fills a gap in the area of detailed 3D shading simulation for PV yield estimation.

State of the field

Existing software can roughly be grouped into three groups: full-suite commercial design tools, simplified yield calculators, and developer libraries.

Full-suite design tools target professional PV planners and combine 3D system modeling, component databases and financial analysis behind a graphical user interface. HelioScope by Aurora Solar is a web-based design platform that approximates near shading by projecting drop shadows from 3D objects rather than performing full ray tracing, and is offered as a paid subscription (Aurora Solar, 2026). PV*SOL premium by Valentin Software is a Windows desktop application that performs detailed sub-module shading analysis at cell level, but is restricted to interactive single-project use (Valentin Software GmbH, 2026). PVsyst is the industry reference for bankable yield reports and additionally provides a separately licensed

40 command-line tool, PVsystCLI, which is the only existing software that combines accurate 3D
 41 near shading with batch automation, but at a cost of several thousand Swiss francs per year
 42 (PVsyst SA, 2026). The System Advisor Model (SAM) developed by NREL is open-source and
 43 exposes a Python wrapper (PySAM), but its 3D shading capabilities are limited compared to
 44 dedicated design tools (National Renewable Energy Laboratory, 2026b).

45 Among the developer libraries, the Python package pvlib (Anderson et al., 2023; Holmgren,
 46 Hansen, & Mikofski, 2018) offers a range of functionalities. However, the rather niche topic of
 47 shading simulation with 3D objects is not included in this package. Another Python-based
 48 software that enables irradiance modeling in two dimensions is pvfactors (Anoma et al., 2017;
 49 Pvfactors, 2022). SoDeLe wraps pvlib in an end-user GUI for quick residential estimates and
 50 likewise omits 3D shading (Steinacker et al., 2026). Web-based tools for solar panel simulations,
 51 such as PVGIS, PVWatts, and RETScreen, provide an accessible means for non-technical
 52 individuals to estimate energy yields based on geographic location and building geometry
 53 (Psomopoulos et al., 2015). However, these tools lack the capability to perform shading
 54 simulations using 3D geometries.

55 Table 1 summarises the tools by the capabilities most relevant to urban-scale PV assessment.
 56 Two patterns emerge. First, every tool that supports detailed 3D shading is either commercial
 57 or restricted to interactive single-project use; the only freely scriptable option, PVsystCLI, is
 58 paywalled. Second, none of the open-source developer libraries can model arbitrary 3D shading,
 59 and none of the surveyed tools is designed for batch processing on a city scale.

Table 1: Comparison of representative PV simulation tools by the capabilities most relevant to automated, city-scale assessment. Prices are approximate annual costs as of early 2026.

Tool	Platform	Price [per year]	Open Source	3D Shading	Interface	Electrical Model	Comp. DB	Source
HelioScope	Web	\$1,900	No	Yes	API	Sub-module	Yes	(Aurora Solar, 2026)
PV*SOL prem.	Desktop	€845	No	Yes	No	Sub-module	Yes	(Valentin Software GmbH, 2026)
PVsyst	Desktop	€758	No	Yes	CLI	Sub-module	Yes	(PVsyst SA, 2026)
SAM	Desktop	Free	Yes	Limited	Py	Diode model	Yes	(National Renewable Energy Laboratory, 2026b)
PVGIS	Web	Free	No	Horizon only	API	Factor	No	(European Commission Joint Research Centre, 2026)
PVWatts	Web	Free	Yes	No	API	Factor	No	(National Renewable Energy Laboratory, 2026a)
SoDeLe	Package	Free	Yes	No	Yes	Diode model	Yes	(Steinacker et al., 2026)
pvlib	Package	Free	Yes	No	Yes	Diode model	Yes	(Anderson et al., 2023; Holmgren, Hansen, & Mikofski, 2018)
pvfactors	Package	Free	Yes	2D only	Yes	None	No	(Anoma et al., 2017; Pvfactors, 2022)

60 simshady now fills two existing gaps: First, it is implemented in typescript and hence

simshady simultaneously enables simulations in the browser and on a local machine. And Second, it takes the calculation intensive task of raytracing for shading simulation with a performant WebGL implementation on the GPU. Since WebGL is not well known in the science community, simshady provides access to this simulation code via better known javascript methods.

Software design

simshady simulates the yield of photovoltaic (PV) systems by considering weather/climate data and shading from local 3D geometry. It is built around three core principles: (1) To run the core simulation code efficiently on the GPU, (2) to expose two different interfaces for both browser-based applications and server-based simulation, and (3) to integrate standardized data models, namely from Three.js and from the Wavefront OBJ file format.

Scene construction and input data

In simshady, a 3D scene is built that represents the environment, comprising primary objects for simulation (e.g., PV panels or target buildings) and surrounding objects that may cast shadows (e.g., neighboring buildings, trees). Weather and climate data are integrated using Global Horizontal Irradiance (GHI) and Direct Normal Irradiance (DNI) datasets, which are reconstructed to include directional irradiance information using the HEALPix framework (Górski et al., 2005; Zonca et al., 2019).

Simulation pipeline

The central ShadingScene class orchestrates the simulation through the following steps:

1. **Geometry pre-processing:** All geometries provided by the user are shifted to the coordinate origin to improve floating-point precision. The simulation mesh is then refined by recursively subdividing triangles whose longest edge exceeds a configurable threshold (default 1.0 m), ensuring a uniform spatial resolution for the irradiance calculation. Reducing this threshold results in a larger number of smaller triangles in the mesh, and therefore a higher resolution of the simulated shading. To improve the simulation speed, a geometry filter removes shading triangles that cannot cast shadows on the simulation geometry given the minimum solar altitude angle present in the irradiance dataset.
2. **Ray tracing:** The simulation utilizes the Möller-Trumbore intersection algorithm (Möller & Trumbore, 1997) to determine if any shading objects obstruct the view between a sky pixel and the main simulation geometry. For each triangle in the simulation geometry, a shading mask is generated, indicating whether an object blocks the line of sight from the sky pixel to the triangle. The shading mask values range from 0 to 1, where 0 indicates that an object shades the triangle, 1 signifies that there is no obstruction and the line of sight is perpendicular to the triangle, and values between 0 and 1 represent cases where there is no obstruction but the angle of incidence is not perpendicular. This computation is fully parallelizable and has been implemented using WebGL.
3. **Irradiance integration and yield calculation:** The radiance values from all sky segments are multiplied by their corresponding shading mask values and summed to obtain the total irradiance received by each triangle. The resulting intensities are converted to electrical yield (in kWh/m²) using a configurable solar-to-electricity conversion efficiency (default 15%). This efficiency includes the PV panel efficiency as well as the ratio of total area and area covered by PV panels.
4. **Visualization:** The computed yield values are normalized and mapped to RGB colors using a configurable colormap (default: viridis). The resulting Three.js mesh carries both the color attribute for visualization and per-triangle intensity attributes for further analysis or export to other formats.

107 **Simshady in the browser**

108 Simshady reuses the data model and objects of Three.js, a common package for 3D web
109 applications. The package processes Three.js meshes and adds the simulated PV yield as a
110 color to the mesh, as shown in Figure 1. Additionally, each triangle of the simulated buildings
111 has its solar yield assigned as an attribute for further processing.



Figure 1: A simulated building with its solar yield, where dark purple represents low yields and light yellow represents high yields. Screenshot taken from ([Openpv.de](https://openpv.de))

112 **Simshady in the terminal**

113 The simshady CLI is a wrapper around the core WebGL-based simulation engine that enables
114 batch processing of photovoltaic-yield analyses on a server. It first parses a small set of required
115 arguments (the simulation geometry and the irradiance data). The supplied geometry files,
116 either JSON objects or Wavefront OBJ files, are handed to a headless Chromium instance
117 launched via Puppeteer ([Google Chrome Team, 2025](https://google.github.io/puppeteer/)).

118 Inside the browser context the full simshady package is injected, the scene is reconstructed, and
119 the GPU-accelerated Möller-Trumbore ray-tracing routine is executed. When the calculation
120 finishes, the CLI extracts the mesh data from the browser and writes the following artefacts into
121 the user-specified output directory: binary files for vertex positions, colors, and intensities; a
122 colour-coded Wavefront OBJ file with per-vertex RGB values; an orthographic top-down PNG
123 snapshot of the scene; and a JSON summary containing per-timestep and total aggregated
124 yield statistics such as total and mean yield or surface area.

125 **Research impact statement**

126 **Tool landscape positioning**

127 simshady together with its CLI occupies a position in the PV-tool landscape that previously
128 did not exist. Two simple matrices, derived from the comparison in Table 1, highlight this.

129 The first matrix groups tools by cost and 3D shading capability. Up to now, every tool that

130 offered full 3D shading of arbitrary geometries was a commercial product, while every free or
131 open-source tool either ignored 3D shading or was restricted to highly simplified geometric
132 assumptions. Figure 2 shows this comparison.

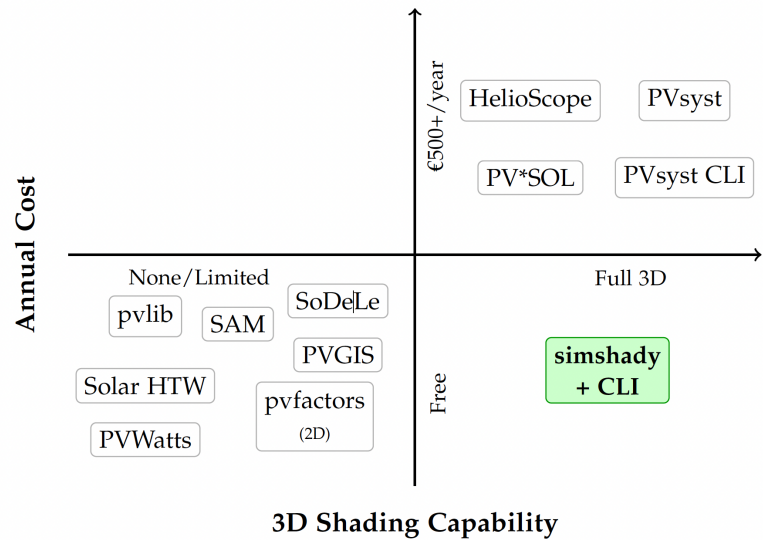


Figure 2: Tool positioning by cost and 3D shading capability. Free tools historically lacked 3D shading; 3D capability required commercial licenses. simshady with its CLI is the first free tool with a full 3D shading capability.

133 The second matrix groups the same tools by simulation scale and 3D shading capability. Tools
134 that can be applied automatically to large numbers of buildings have so far skipped 3D shading
135 entirely; tools with accurate 3D shading have been built around interactive single-project
136 workflows. Figure 3 shows this comparison.

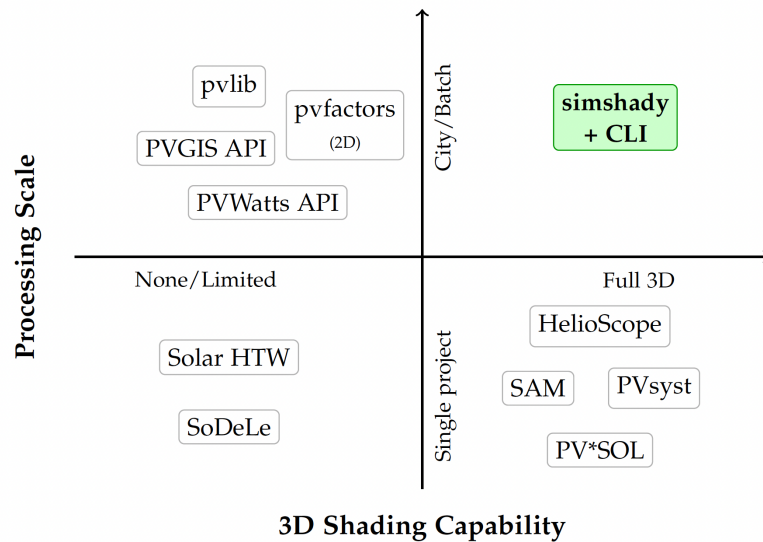


Figure 3: Tool positioning by simulation scale and 3D shading capability. City-scale PV assessment with accurate 3D shading was not possible before simshady and its CLI.

137 In both matrices the cell that simshady now occupies was empty before this work. simshady
138 is therefore the first free and open-source tool with full 3D shading of arbitrary geometries,
139 and the first such tool designed to handle both single buildings or entire cities.

140 City-scale demonstration: Munich

141 To show that this position is more than theoretical, the CLI was used to simulate the PV yield
142 for the entire city of Munich. The original data consists of 109 CityGML LoD2 tiles. Each tile
143 had a size of $2 \times 2 \text{ km}^2$. For processing purposed each tiles was split into four tiles, one square
144 kilometre each. The data contains roughly 200 km^2 of building surface in total. Irradiance
145 data from the national solar radiation data base (Sengupta et al., 2018) was projected onto a
146 HEALPix sky and ray-traced against the full geometry. The same CLI workflow that any user
147 can apply to their own input data produced per-triangle yield artefacts for every tile: binary
148 vertex/colour/intensity buffers, vertex-coloured OBJ meshes, orthographic top-down PNG
149 snapshots, and JSON yield summaries. Figure 4 shows the 3D mesh output for one tile, and
150 Figure 5 shows the orthographic top-down PNG that the CLI generates for the full city.

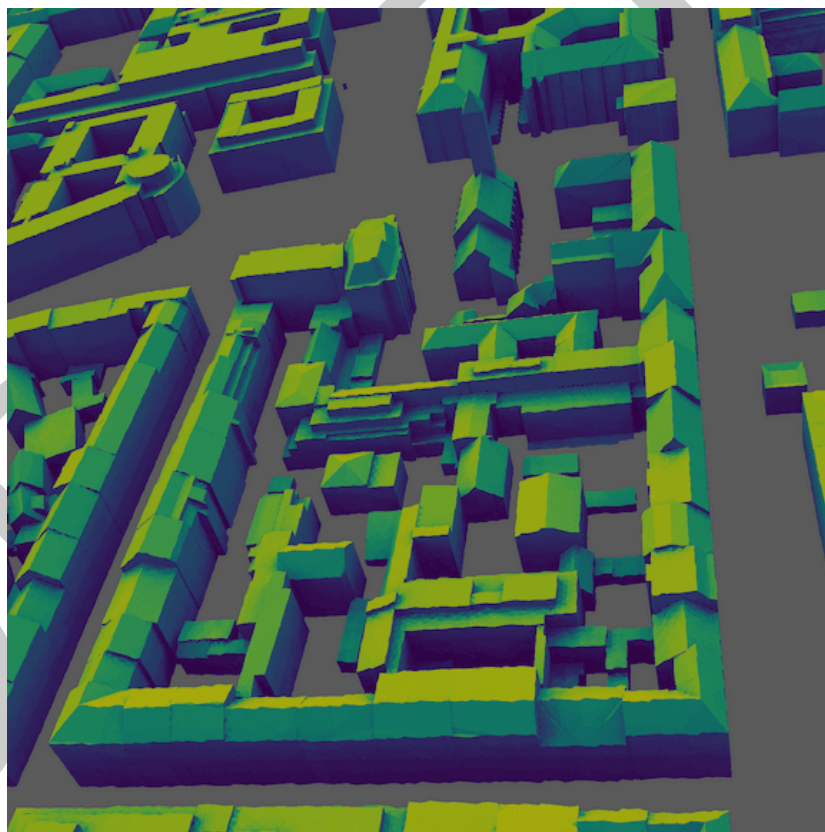


Figure 4: 3D mesh output of the CLI for a single Munich tile, with per-triangle annual PV yield mapped onto the building geometry (dark blue = low, light yellow = high).

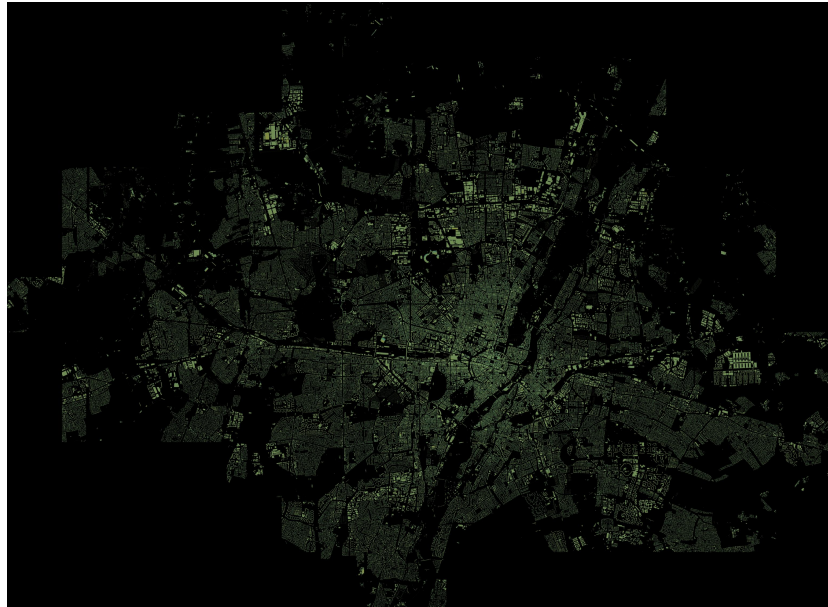


Figure 5: Orthographic top-down snapshots generated by the CLI for all Munich tiles, combined in one image.

151 The complete simulation took roughly 110 hours on a single cloud GPU server running four
152 parallel CLI instances and cost less than 60 USD. The full output, including geometry, intensities,
153 metadata and visualisation artefacts for all tiles, is openly available as a dataset on Zenodo
154 under CC BY 4.0 ([Estrella & Kotthoff, 2026](#)).

155 Conclusion

156 The simshady package serves two primary purposes: it provides a solution for scientific
157 calculations of PV yield, while also facilitating science communication through interactive and
158 user-friendly simulations. This eliminates the need for specialized software or programming
159 knowledge, making it accessible to a broader range of users. Furthermore, by implementing the
160 main algorithm in WebGL, the package achieves higher performance than a pure Javascript
161 implementation, and it offers a JavaScript wrapper around PV simulation in WebGL. This is
162 particularly beneficial because WebGL is a language that is not widely known among scientists,
163 and thus can be challenging for them to implement their own code, making the simshady
164 package a valuable tool for simplifying this process.

165 Credit Authorship Statement

166 FK: Conceptualization, Software, Funding acquisition, Writing – original draft
167 MG: Conceptualization, Software, Funding acquisition, Writing – review & editing
168 KH: Conceptualization, Software, Funding acquisition, Writing – review & editing
169 ME: Software, Writing – review & editing
170 KP: Conceptualization, Software, Funding acquisition, Writing – review & editing

171 AI usage disclosure

172 After designing the software architecture and writing core functionalities, both open-weight and
 173 properitarian LLM-based chatbots where used to implement or debug some of the methods in
 174 simshady. Generative agentic AI tools that can write and edit code autonomously were not
 175 used. The open-weight models gpt-oss:120b and deepseek-r1:70b were used for reviewing the
 176 written text of this paper.

177 Acknowledgements

178 The development of this software was funded by the German Federal Ministry of Research,
 179 Technology and Space within the “Software Sprint - Support for Open Source Developers”
 180 program by Prototype Fund.

181 References

- 182 Anderson, K. S., Hansen, C. W., Holmgren, W. F., Jensen, A. R., Mikofski, M. A., & Driesse,
 183 A. (2023). Pvlb python: 2023 project update. *Journal of Open Source Software*, 8(92),
 184 5994. <https://doi.org/10.21105/joss.05994>
- 185 Anoma, M. A., Jacob, D., Bourne, B. C., Scholl, J. A., Riley, D. M., & Hansen, C. W.
 186 (2017). View factor model and validation for bifacial PV and diffuse shade on single-
 187 axis trackers. *2017 IEEE 44th Photovoltaic Specialist Conference (PVSC)*, 1549–1554.
 188 <https://doi.org/10.1109/PVSC.2017.8366704>
- 189 Aurora Solar. (2026). *HelioScope solar design software*. <https://helioscope.aurorasolar.com/>.
- 190 Estrella, M., & Kotthoff, F. (2026). *simshady munich PV yield simulation dataset (CityGML*
 191 *LoD2, 432 tiles)*. Zenodo. <https://doi.org/10.5281/zenodo.18742162>
- 192 European Commission Joint Research Centre. (2026). *PVGIS – photovoltaic geographical*
 193 *information system*. https://re.jrc.ec.europa.eu/pvg_tools/en/.
- 194 Google Chrome Team. (2025). *Puppeteer: Headless chrome node.js API* (Version v24.31.0).
 195 Google. <https://github.com/puppeteer/puppeteer>
- 196 Górski, K. M., Hivon, E., Banday, A. J., Wandelt, B. D., Hansen, F. K., Reinecke, M., &
 197 Bartelmann, M. (2005). HEALPix: A framework for high-resolution discretization and
 198 fast analysis of data distributed on the sphere. *The Astrophysical Journal*, 622(2), 759.
 199 <https://doi.org/10.1086/427976>
- 200 Holmgren, W. F., Hansen, C. W., & Mikofski, M. A. (2018). Pvlb python: A python package
 201 for modeling solar energy systems. *Journal of Open Source Software*, 3(29), 884.
- 202 Holmgren, W. F., Hansen, C. W., Stein, J. S., & Mikofski, M. A. (2018). Review of open
 203 source tools for PV modeling. *2018 IEEE 7th World Conference on Photovoltaic Energy*
 204 *Conversion (WCPEC)(a Joint Conference of 45th IEEE PVSC, 28th PVSEC & 34th EU*
 205 *PVSEC)*, 2557–2560. <https://doi.org/10.1109/PVSC.2018.8548231>
- 206 International Energy Agency. (2023). *Tripling renewable power capacity by 2030 is vital to*
 207 *keep the 1.5°C goal within reach*. [https://www.iea.org/commentaries/tripling-renewable-](https://www.iea.org/commentaries/tripling-renewable-power-capacity-by-2030-is-vital-to-keep-the-150c-goal-within-reach)
 208 [power-capacity-by-2030-is-vital-to-keep-the-150c-goal-within-reach](https://www.iea.org/commentaries/tripling-renewable-power-capacity-by-2030-is-vital-to-keep-the-150c-goal-within-reach).
- 209 Jakica, N. (2018). State-of-the-art review of solar design tools and methods for assessing
 210 daylighting and solar potential for building-integrated photovoltaics. *Renewable and*
 211 *Sustainable Energy Reviews*, 81, 1296–1328. <https://doi.org/10.1016/j.rser.2017.05.080>
- 212 Kumar Vashishtha, V., Yadav, A., Kumar, A., & Kumar Shukla, V. (2022). An overview of
 213 software tools for the photovoltaic industry. *Materials Today: Proceedings*, 64, 1450–1454.

- 214 <https://doi.org/10.1016/j.matpr.2022.04.737>
- 215 Möller, T., & Trumbore, B. (1997). Fast, minimum storage ray-triangle intersection. *Journal*
216 *of Graphics Tools*, 2(1), 21–28. <https://doi.org/10.1080/10867651.1997.10487468>
- 217 National Renewable Energy Laboratory. (2026a). *PVWatts calculator*. [https://pvwatts.nrel.](https://pvwatts.nrel.gov)
218 [gov](https://pvwatts.nrel.gov).
- 219 National Renewable Energy Laboratory. (2026b). *System advisor model (SAM)*. [https:](https://sam.nrel.gov)
220 [//sam.nrel.gov](https://sam.nrel.gov).
- 221 *Openpv.de*. www.openpv.de.
- 222 Psomopoulos, C. S., Ioannidis, G. C., Kaminaris, S. D., Mardikis, K. D., & Katsikas, N.
223 G. (2015). A comparative evaluation of photovoltaic electricity production assessment
224 software (PVGIS, PVWatts and RETScreen). *Environmental Processes*, 2, 175–189.
225 <https://doi.org/10.1007/s40710-015-0092-4>
- 226 *Pvfactos: Open-source view-factor model for diffuse shading and bifacial PV modeling* (Version
227 1.5.2). (2022). <https://github.com/SunPower/pvfactos>
- 228 PVsyst SA. (2026). *PVsyst photovoltaic software*. <https://www.pvsyst.com>.
- 229 Sengupta, M., Xie, Y., Lopez, A., Habte, A., Maclaurin, G., & Shelby, J. (2018). The national
230 solar radiation data base (NSRDB). *Renewable and Sustainable Energy Reviews*, 89, 51–60.
- 231 Steinacker, H., Mucke, J., Ott, E., & Stickel, M. (2026). *SoDeLe: Solarsimulation denkbar*
232 *leicht*. <https://github.com/QuaSi-Software/SoDeLe>.
- 233 Valentin Software GmbH. (2026). *PV*SOL premium – planning and simulation of photovoltaic*
234 *systems*. <https://valentin-software.com/en/products/pvsol-premium/>.
- 235 Zonca, A., Singer, L., Lenz, D., Reinecke, M., Rosset, C., Hivon, E., & Gorski, K. (2019).
236 Healpy: Equal area pixelization and spherical harmonics transforms for data on the sphere
237 in python. *Journal of Open Source Software*, 4(35), 1298. [https://doi.org/10.21105/joss.](https://doi.org/10.21105/joss.01298)
238 [01298](https://doi.org/10.21105/joss.01298)